

Universidad San Sebastián



FACULTAD DE INGENIERÍA Y TECNOLOGÍA

Carrera de Ingeniería Civil Informática

Informe Técnico Final:

Proyecto USS SpiderBot — Robot Cuadrúpedo de

Reconocimiento

Integrantes:

Moisés Amundarain

Camila Aravena

Fernando Ramírez

Juan Pablo Vargas

María José Santander

Puerto Montt, Chile

23 de junio de 2026

Índice

Resumen Ejecutivo	2
1. Introducción y Objetivos	3
1.1. Introducción	3
1.2. Objetivos	3
1.2.1. Objetivo General	3
1.2.2. Objetivos Específicos	3
2. Diseño Mecánico y Estructural (OpenSCAD)	5
2.1. Estructura del Chasis (Doble Deck)	5
2.2. Articulaciones y Extremidades (Fémur y Tibia)	5
2.3. Precisión en Tolerancias e Interfaces Mecánicas	6
2.3.1. Cavidades de Servomotores (Snug Fit)	6
2.3.2. Bolsillos de Doble Profundidad (Dual-Depth Pocket)	7
2.3.3. Cerraduras de Acople Horn (Single-Arm Horn Locks)	7
3. Arquitectura de Hardware y Conexiones	8
3.1. Lista de Componentes (BOM)	8
3.2. Aislamiento de Potencia y Referencias GND	8
3.3. Tabla de Direccionamiento I2C y GPIOs	9
4. Desarrollo del Firmware (MicroPython)	10
4.1. Arquitectura Multitarea Asíncrona (uasyncio)	10
4.2. Algoritmo de Locomoción: Marcha de Gateo (Crawl Gait)	10
4.3. Lazo de Compensación Inercial Activa	11
4.4. Evasión Reactiva y Servidor HTTP con Dashboard	12
5. Simulación y Laminación para Impresión 3D	13
5.1. Simulación del Circuito en Wokwi	13
5.2. Parámetros de Laminación FDM (Creality Print)	13
Conclusiones y Trabajo Futuro	15

Resumen Ejecutivo

El presente informe detalla el diseño, modelado mecánico, desarrollo de hardware y arquitectura de software del **USS SpiderBot**, un robot cuadrúpedo de 8 grados de libertad (8-DoF) optimizado para la exploración de terrenos de catástrofe y zonas obstruidas por escombros.

El robot integra un chasis paramétrico desarrollado íntegramente en **OpenSCAD** y optimizado para la impresión 3D mediante modelado por deposición fundida (FDM). En el apartado electrónico, la plataforma de control está gobernada por un microcontrolador **ESP32 DevKit V1** ejecutando firmware en **MicroPython**. El bus de señales físico del bus I2C está compartido de forma concurrente por un expansor PWM de 12 bits **PCA9685** y una unidad de medida inercial (IMU) **MPU6050**.

El firmware implementa un lazo cerrado de control inercial proporcional que corrige dinámicamente la inclinación del chasis en tiempo real inyectando compensaciones angulares en las articulaciones de apoyo. El robot cuenta con evasión reactiva de obstáculos mediante un sensor de ultrasonidos **HC-SR04** y expone un servidor web asíncrono que aloja un dashboard de telemetría inercial y mandos remotos interactivos mediante una interfaz con estética Glassmorphism. El sistema completo fue validado de forma interactiva en la plataforma de simulación **Wokwi** y su viabilidad física fue verificada mediante perfiles de laminación y optimización de tolerancias mecánicas.

1 Introducción y Objetivos

1.1 Introducción

La robótica móvil en terrenos no estructurados representa uno de los desafíos más significativos para la ingeniería de automatización moderna. A diferencia de los sistemas de tracción por ruedas u orugas, los robots con locomoción basada en patas ofrecen una versatilidad superior al poder sortear de forma directa obstáculos discretos, ascender escaleras y nivelarse de manera activa sobre superficies irregulares o inclinadas.

El proyecto **USS SpiderBot** consiste en el diseño e implementación de un prototipo de robot cuadrúpedo autónomo con estabilidad inercial de lazo cerrado, sensor de evasión de colisiones y panel de control web. Este desarrollo ha sido realizado como parte del entregable final de la asignatura **Taller de Programación I** (Universidad San Sebastián), integrando conceptos de desarrollo físico estructurado, programación paramétrica en CAD, lógica en sistemas embebidos de tiempo real y desarrollo web nativo para el operador de rescate.

1.2 Objetivos

1.2.1 Objetivo General

Diseñar, validar y documentar un robot cuadrúpedo articulado de 8 grados de libertad mediante modelado OpenSCAD, control embebido MicroPython y simulación interactiva, cumpliendo con los estándares mecánicos de holgura para impresión 3D y estabilidad de locomoción autónoma.

1.2.2 Objetivos Específicos

1. Desarrollar el chasis paramétrico y extremidades en OpenSCAD, aplicando tolerancias de precisión para impresión FDM y cerraduras chavetadas para los horns de los servos.
2. Implementar la arquitectura electrónica unificada compartiendo la señal I2C entre el módulo PCA9685 y el acelerómetro MPU6050, aislando la potencia mediante un UBEC step-down.

3. Programar el lazo cerrado de nivelación activa inercial y las corrutinas de locomoción del ciclo de marcha de gateo (Crawl Gait) mediante concurrencia cooperativa asíncrona.
4. Generar un panel web interactivo autocontenido alojado en la ESP32 que grafique telemetría inercial y reciba órdenes por teclado y mouse de forma no bloqueante.
5. Validar el circuito y la compatibilidad lógica del firmware en la plataforma de simulación Wokwi y preparar el archivo del laminador Crealty Print para fabricación física.

2 Diseño Mecánico y Estructural (OpenSCAD)

El modelado tridimensional del USS SpiderBot se basó en el paradigma de diseño de geometría sólida constructiva (CSG) mediante el uso de código paramétrico en OpenSCAD.

2.1 Estructura del Chasis (Doble Deck)

El cuerpo central del robot consta de un sistema de doble placa circular (doble deck) que aísla los componentes de potencia y cableado del plano de los sensores y control:

- **Placa Inferior:** Un cilindro plano de radio $r = 65$ mm y espesor $t = 3$ mm que aloja perimetralmente los 4 brackets de fijación de servos de cadera. Cuenta con ranuras pasantes para la fijación de la batería mediante correas de velcro.
- **Placa Superior:** Placa de protección que se acopla a la inferior mediante 4 pilares espaciadores M3 de 19 mm de alto. Incorpora una cuna central de dimensiones $84 \times 56 \times 3$ mm para alojar a presión la protoboard de 400 puntos, una cuna trasera para el regulador XL6009E1 y 4 agujeros de 14 mm de diámetro para el ruteo de cables.
- **Soporte HC-SR04:** Soporte vertical de 2 mm de espesor con dos orificios de 16.5 mm de diámetro espaciados a 26 mm entre centros, reforzado por dos gussets triangulares laterales para resistir fuerzas de cizalladura inerciales en caso de impacto.

2.2 Articulaciones y Extremidades (Fémur y Tibia)

La configuración del robot se define como **Pitch-Pitch**, lo que significa que las articulaciones de la cadera (Coxa) y la rodilla (Fémur) rotan sobre ejes horizontales y paralelos.

- **Fémur:** Brazo de articulación de 55 mm de longitud entre centros. Une la corona del servo de cadera con el cuerpo del servo de rodilla. Se utilizó la función `hull()` para conectar suavemente la corona cilíndrica con la cavidad prismática del servo de rodilla, optimizando la distribución de masa y la resistencia a la flexión.



Figura 1: Renderizado 3D paramétrico del chasis final y las extremidades articuladas del USS SpiderBot.

- **Tibia:** Pierna inferior curva con una longitud vertical de 65 mm y pie semiesférico de radio $r = 5.5$ mm. La curvatura estructural del modelo aumenta el despeje libre sobre el suelo y distribuye las fuerzas de reacción del impacto del pie en la marcha.

2.3 Precisión en Tolerancias e Interfaces Mecánicas

2.3.1 Cavidades de Servomotores (Snug Fit)

Debido a la contracción del plástico durante el enfriamiento de la extrusión FDM, se definió una tolerancia de precisión de **0.3 mm totales** (0.15 mm por lado) para las cavidades de los servos SG90/MG90S. Sus dimensiones internas se establecieron en 24.2 x 23.2 x 13.4 mm, logrando un ajuste a presión sin juego mecánico. Las ranuras laterales de las bridas se fijaron a 2.8 mm de ancho y los agujeros de roscado autorroscante de tornillos M2 a 2.0 mm de diámetro.

2.3.2 Bolsillos de Doble Profundidad (Dual-Depth Pocket)

Para evitar el estrangulamiento de los cables de salida en la base del servo, se implementaron bolsillos de doble profundidad:

$$D(z) = \begin{cases} 30.0 \text{ mm} & \text{si } z \leq 5.0 \text{ mm} \quad (\text{Espacio para cables}) \\ 25.0 \text{ mm} & \text{si } z > 5.0 \text{ mm} \quad (\text{Ajuste del cuerpo}) \end{cases} \quad (1)$$

Esto garantiza que los tornillos autorroscantes M2 plásticos agarren sobre plástico 100 % macizo de los brackets a $X = \pm 14.25 \text{ mm}$.

2.3.3 Cerraduras de Acople Horn (Single-Arm Horn Locks)

Para eliminar las pérdidas de transmisión por rozamiento o juego entre los acoples, se diseñaron chaveteros de un brazo (*single-arm horn locks*) embutidos en las caras de acople mecánico del fémur ($X = 16.0, Y = 5.2, Z = 2.6 \text{ mm}$) y la tibia ($X = 5.2, Y = 16.0, Z = 2.6 \text{ mm}$), bloqueando mecánicamente el brazo del horn estriado original del servo en el sólido de plástico.

3 Arquitectura de Hardware y Conexiones

3.1 Lista de Componentes (BOM)

La Tabla 1 muestra la lista de materiales requerida para el montaje del prototipo físico:

Cuadro 1: Lista de Materiales del USS SpiderBot

Componente	Función / Propósito	Cantidad
ESP32 DevKit V1 (38 pines)	Procesamiento central, WiFi y Servidor Web	1
Módulo PCA9685 I2C	Expansor PWM de 12-bit para los 8 servos	1
Servomotores SG90 / MG90S	Actuadores de cadera y rodilla (Coxa/Fémur)	8
IMU MPU6050 (GY-521)	Acelerómetro y Giroscopio de 6 ejes	1
Sensor Ultrasonido HC-SR04	Sensor de proximidad frontal	1
Buzzer Activo 5V	Señales acústicas y alarma	1
UBEC Step-down 5V / 3A	Aislamiento de potencia para servomotores	1
Batería Li-ion 2S 7.4V	Fuente de energía principal	1

3.2 Aislamiento de Potencia y Referencias GND

Los servomotores de micro-robótica como los SG90 operan típicamente a 5V de corriente continua y pueden demandar hasta 500 mA cuando levantan la carga estática del chasis.

- **Alimentación de Control:** El ESP32 se alimenta mediante la salida regulada de 5V del regulador UBEC conectado al pin *Vin*. El ESP32 a su vez entrega 3.3V estables a la IMU MPU6050 y a la lógica digital del driver PCA9685.
- **Alimentación de Fuerza:** El canal de potencia del UBEC (5V / 3A continuos) se conecta de forma dedicada a la bornera azul de tornillo de entrada de fuerza del PCA9685 (*V+* y *GND*).
- **Unificación de Referencias:** Es un requisito indispensable unificar todos los terminales de tierra común (*GND* lógico del PCA9685, *GND* del regulador UBEC, *GND* del ESP32, *GND* de la IMU y del sonar) en un solo nodo físico para evitar bucles de masa y deriva de tensión.

3.3 Tabla de Direccionamiento I2C y GPIOs

El expansor PCA9685 y la IMU MPU6050 comparten de forma física el bus de comunicación serie I2C, direccionando las llamadas mediante sus identidades en el bus de datos (0x40 para el driver y 0x68 para el sensor inercial).

Cuadro 2: Mapa de Conexiones Físicas de la ESP32

Dispositivo	Pin Dispositivo	Pin ESP32	Tipo de Señal
PCA9685 (PWM)	SDA / SCL	GPIO 21 / GPIO 22	I2C (Bus Compartido)
MPU6050 (IMU)	SDA / SCL	GPIO 21 / GPIO 22	I2C (Bus Compartido)
HC-SR04 (Sonar)	TRIG / ECHO	GPIO 18 / GPIO 19	Entrada/Salida Digital
Buzzer Activo	SIG (+)	GPIO 14	Salida Digital
Todos los Servos	Canales 0–7	Salidas PCA9685	PWM de alta resolución

4 Desarrollo del Firmware (MicroPython)

La programación del microcontrolador se basó en un diseño multitarea cooperativo no bloqueante que garantiza un tiempo de respuesta inferior a 25 ms para tareas críticas.

4.1 Arquitectura Multitarea Asíncrona (`uasyncio`)

Para evitar el uso de funciones de retardo bloqueantes como `time.sleep_ms()`, las cuales congelarían el procesamiento del servidor web y del bucle inercial, se implementaron corrutinas concurrentes coordinadas por el planificador asíncrono `uasyncio`:

- `sensor_updater()`: Lee periódicamente el acelerómetro MPU6050 y el sonar HC-SR04 a una frecuencia de refresco de 20 Hz (50 ms).
- `locomotion_loop()`: Supervisa el estado de comandos globales de movimiento y actualiza las posiciones angulares de los servos.
- `start_server_task()`: Ejecuta el socket TCP asíncrono que atiende las peticiones HTTP del Dashboard.

4.2 Algoritmo de Locomoción: Marcha de Gateo (Crawl Gait)

Para mantener la estabilidad estática, el robot debe asegurar en todo momento que la proyección ortogonal del centro de masa de su cuerpo se sitúe dentro de la superficie encerrada por las patas de apoyo (el polígono de sustentación). El ciclo de marcha implementado consta de **5 fases discretas**:

1. **Fase de Desplazamiento Corporal (Stance Shift)**: Con las 4 patas apoyadas en el suelo, los servos Coxa giran hacia atrás al unísono, empujando el chasis hacia adelante.
2. **Fase Swing FR**: La pata delantera derecha (FR) se levanta a 90° de fémur, avanza su Coxa a 110° y vuelve a descender al ángulo de apoyo (60°).
3. **Fase Swing RR**: La pata trasera derecha (RR) repite el ciclo levantando rodilla, adelantando cadera y posando el pie en el suelo.

4. **Fase Swing FL:** La pata delantera izquierda (FL) ejecuta la oscilación (swing) adelantando cadera a 75° y posando a 120° .
5. **Fase Swing RL:** La pata trasera izquierda (RL) completa el ciclo secuencial de marcha.

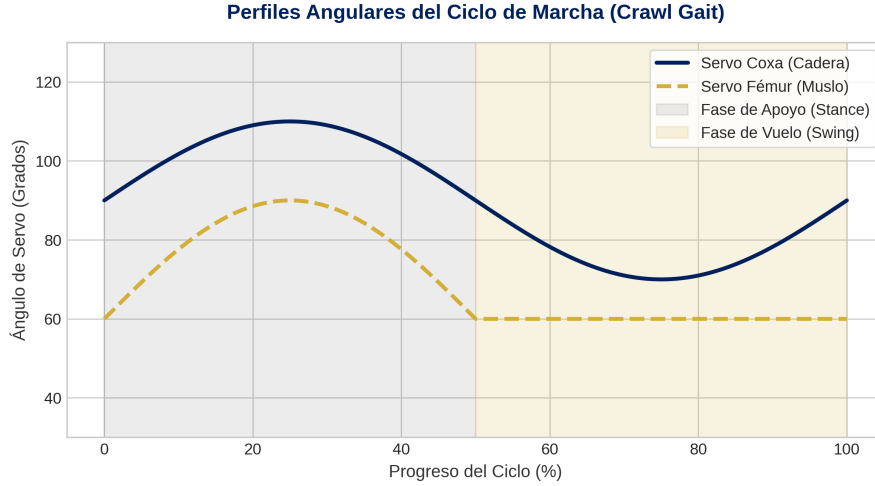


Figura 2: Curvas de comportamiento angular para las articulaciones de cadera (Coxa) y muslo (Fémur) simuladas durante la marcha de gateo.

4.3 Lazo de Compensación Inercial Activa

La nivelación de lazo cerrado se ejecuta en la corrutina de locomoción. Utiliza la lectura de los ángulos calculados de Pitch (θ_p) y Roll (θ_r) promediados del giroscopio y el acelerómetro de la MPU6050. Si el valor absoluto de la inclinación supera la banda de tolerancia de $\pm 3^\circ$, se inyecta una compensación proporcional a los servos de fémur de las patas en fase de apoyo:

$$\theta_{compensado} = \theta_{base} \pm (K_p \cdot \theta_{sensor}) \quad (2)$$

Donde $K_p = 1.2$ es la constante proporcional obtenida tras la calibración. Esto eleva los fémures del lado caído y baja los del lado elevado, manteniendo la horizontalidad del chasis. Las patas que están en el aire (swing) se excluyen para evitar perturbaciones de avance.

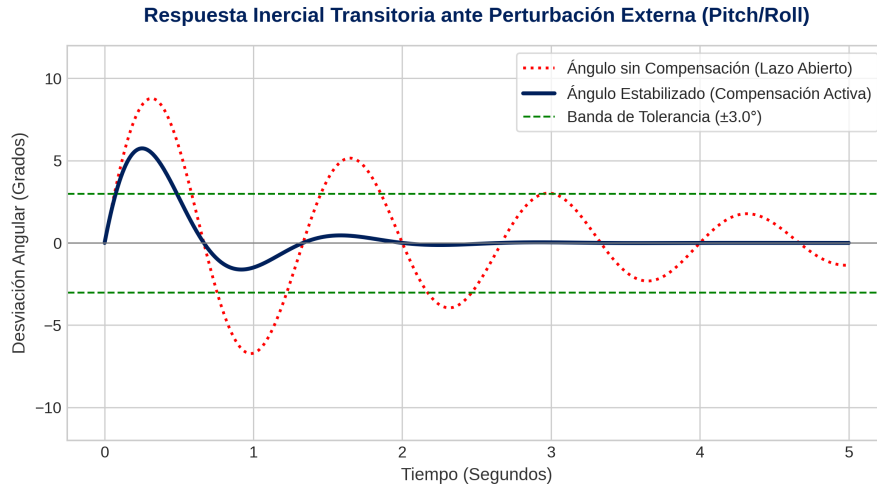


Figura 3: Simulación en Python de la respuesta del sensor inercial bajo compensación activa frente a lazo abierto.

4.4 Evasión Reactiva y Servidor HTTP con Dashboard

- **Seguridad de Lazo Abierto:** Si la lectura de distancia de la corrutina del sonar es inferior a 15 cm, se interrumpe la locomoción de inmediato, se desactivan los servos enviándolos a la pose de reposo estática (`pos_reposo()`) y se activa una oscilación acústica intermitente a 2 kHz en el buzzer.
- **Dashboard Glassmorphism Embebido:** Se diseñó una interfaz gráfica completa en HTML, CSS Glassmorphism y JavaScript nativo. Dado que la ESP32 tiene recursos de memoria limitados, el servidor lee el archivo `index.html` por bloques de 512 bytes y los envía al socket sin saturar la pila de memoria dinámica. El dashboard incluye un monitor inercial vectorial SVG y recibe mandos de teclado no bloqueantes para el control de locomoción.

5 Simulación y Laminación para Impresión 3D

5.1 Simulación del Circuito en Wokwi

Para validar la lógica del firmware antes del cableado real, se diseñó el circuito completo en Wokwi:

- **Control de Fallback de Servos Directo:** Al no contar Wokwi con simulación física del chip PCA9685 por bus I2C de forma estándar en MicroPython, se programó un fallback dinámico en `'main.py'`. Si la inicialización I2C a la dirección `0x40` falla, el controlador conmuta automáticamente a la clase `'ESP32ServoDirect'` la cual implementa la modulación PWM directa usando `'machine.PWM'` nativo en los GPIOs de control de la ESP32.
- **AP de Conexión Virtual:** La placa ESP32 del simulador se conecta de forma automática a la red pública `'Wokwi-GUEST'` en modo estación (STA). Esto permite abrir la IP de la consola directamente en el navegador local, enrutando los comandos remotos hacia el simulador virtual.

5.2 Parámetros de Laminación FDM (Creality Print)

Para fabricar las piezas estructurales paramétricas garantizando rigidez contra impactos y estabilidad en las uniones roscadas plásticas, se definieron los siguientes parámetros en Creality Print:

- **Altura de capa:** 0.2 mm (balance ideal entre tiempo y adherencia de capas).
- **Recuento de paredes (perímetros):** Mínimo 4 perímetros. Esto proporciona un bloque macizo de plástico alrededor de los agujeros M2 del chasis y las articulaciones, evitando el barrido de rosca al apretar los tornillos.
- **Relleno (Infill):** 30 % en patrón **Gyroid** (giroidal). El patrón giroidal ofrece resistencia torsional tridimensional isotrópica, ideal para extremidades robóticas sometidas a esfuerzos dinámicos.
- **Soportes:** Excluidos de las cavidades mediante orientación óptima de las piezas en el plato, manteniendo la precisión dimensional de los slots de servos.

El proyecto de laminación completo fue agrupado y guardado en el archivo [spiderbot.3mf](file:///mnt/0407-4e80-b8af-5417ffbdee8e/Github/Taller

Conclusiones y Trabajo Futuro

Conclusiones

El proyecto **USS SpiderBot** ha permitido integrar con éxito múltiples disciplinas de la ingeniería de software y hardware:

1. El uso de modelado paramétrico en OpenSCAD con control riguroso de holguras (0.3 mm total) y bolsillos dual-depth demostró ser una solución fiable para evitar errores de fabricación FDM comunes como el sobredimensionamiento de orificios o la colisión mecánica en uniones móviles.
2. La arquitectura de software basada en `uasyncio` permitió resolver el cuello de botella del pooling y de los tiempos de espera bloqueantes en MicroPython, logrando un control coordinado que opera el servidor web, el sensor ultrasónico y la estabilización inercial de forma fluida.
3. El lazo cerrado proporcional inercial proporciona estabilidad estática y dinámica activa, lo que permite al cuadrúpedo adaptarse a desniveles físicos de forma autónoma.

Trabajo Futuro

Para extender el alcance actual del proyecto y preparar al robot para aplicaciones reales de rescate, se proponen las siguientes mejoras:

- **Cinemática Inversa Completa (IK):** Migrar el control cinemático actual (Pitch-Pitch) a un sistema 3-DoF por pata con servomotor de aducción/abducción. Esto permitiría describir trayectorias circulares o elípticas exactas del pie en el espacio 3D, facilitando giros sobre el eje vertical sin patinar y reduciendo las corrientes transitorias.
- **Integración de Cámara y Modelo de Búsqueda:** Montar una cámara ESP32-CAM conectada a una red de inferencia de visión computacional local (como TensorFlow Lite Micro) para la detección automática de señales de emergencia, luces o rostros en zonas de desastre.

- **Marchas Dinámicas (Trot Gait):** Programar marchas dinámicas de trote alternado con fases de vuelo para aumentar la velocidad de avance del chasis en terrenos llanos.